

## AI Engineering

# Agentic Dependency Parsing in Low-Resource Settings

Dang Nhat Quang, Han Dai, Nikhila Gadge, Jan Raring

Submission: 19 July 2026

### Abstract

In natural language processing (NLP), dependency parsing refers to the task of constructing sentence-level dependency trees. Often, the resulting dependency–head relations are annotated to specify the type of dependency. Large collections of such dependency trees—so called *treebanks*—are a valuable resource in the study of syntax, grammar, and typology. The manual creation of such dependency trees, however, is a very laborious undertaking which should, if possible, be performed at least semi-automatically. For high-resource languages, many popular NLP libraries (Stanza, NLTK, SpaCy) provide the necessary tooling for exactly this sort of (pre-)annotation. In the case of low-resource languages, however, such tools are typically not available. This observation has instigated us to test whether an AI agent, i.e. an LLM equipped with adequate tools, can function as a general purpose dependency parser, thereby eliminating the need to train purpose-built neural networks as used by the aforementioned libraries.

### 1 Introduction

In natural language processing (NLP), dependency parsing refers to the task of constructing sentence-level dependency trees. Syntactically, a dependency tree is a directed, connected, acyclic graph with words as nodes and *dependency-head*-relations as edges. Semantically, it reflects a sentence’s dependency structure: Each word is linked to the head of the phrase that it is part of. There is always exactly one word that does not itself have a head. This word is called the *root* of the sentence. For additional specification, the edges of such a dependency tree are often annotated with a label taken from a pre-defined set of possible dependency relations.

Large collections of such dependency trees—so called *treebanks*—are being used in a number of linguis-

tic subfields: In syntax they shed light on the possible sentence structures a language allows for; in grammar they help explain to what sort of agreement constraints words in a phrase are subjected to; and in linguistic typology, languages are classified and compared based on the prevailing patterns in word order.

One popular guideline for dependency annotation is the *Universal Dependencies* (UD) framework. Its main proposition lies in the specification of conventions and tag sets for the annotation of parts of speech, morphological features, and dependency relations that are universal enough to be applicable to virtually all natural languages. Currently, the official UD collection contains 353 *treebanks* for 193 different languages. UD also defines the CoNLL-U file format used for storing

annotated sentences and entire treebanks along with metadata in a machine-readable manner. In this format, sentences are represented as tables with one row for every token/word and ten columns holding the annotations: ID, FORM, LEMMA, UPOS, XPOS, FEATS, HEAD, DEPREL, DEPS, and MISC. For now it suffices to know that ID is an integer index starting at 1, and that HEAD stores the ID of the word’s head or 0 if it is the root. The type of dependency relation can be specified in DEPREL and DEPS.

The creation of a treebank like this typically requires a lot of manual labor. To reduce the work load as much as possible, many projects opt for a semi-automated approach, using a pre-trained tagger for intermediate annotations that, in a second step, get corrected by a human. For high-resource languages, taggers are available via all major NLP libraries (Stanza, SpaCy, NLTK), but for the majority of the world’s roughly 7000 languages no such tools exist.

As the training of a tagger itself postulates the existence of a treebank to serve as training data, the problem of not having a pre-trained model is not one to be resolved easily. Additional challenges stem from the fact that many low resource languages have not undergone standardization and so the little amount of data that might be available displays high entropy, causing the training signal to be quite noisy. This throws annotators back into the situation of having to do everything by hand, which—depending on the availability of funding—can make the creation of a new treebank simply unaffordable.

But there is hope: The success of large language models (LLMs) as universal problem solvers gives us good reason to believe that the aforementioned bottle neck relating to manual labeling is indeed surmountable. More specifically, hundreds of treebanks can be assumed to be part of the data used for model training and even if a particular low-resource language is largely out of distribution, studies on cross-lingual transfer have shown that LLMs are reasonably good at extrap-

olating knowledge of a higher-resource language to a closely related low-resource language.

What makes dependency parsing nonetheless challenging is the topology of the problem: Dependency trees are inherently non-linear and dependency relations can stretch over long distances—conditions that LLMs are known to be less good at. The aim of this project is to measure the performance of smaller LLMs in the 7B–12B range on this task and to see if providing the model with different tools for example-retrieval, morphological analysis and lexicon look-ups has an impact on the output quality.

## 2 Related Work

While all major Python NLP libraries supply dependency parser components, their approaches are quite different.

**SpaCy.** The standard `DependencyParser` component in SpaCy is transition-based and uses a variation of the non-monotonic arc-edger transition system developed by Honnibal & Johnson (2014). It learns labelled dependency parsing jointly with sentence segmentation. Modifications are made to enable the generation of non-projective parses (Explosion AI, 2026a). According to their official documentation, SpaCy currently achieves an unlabeled attachment score of over 95% on the Penn Treebank, an English-only treebank that is, however, not directly comparable to Universal Dependencies (Explosion AI, 2026b).

**Stanza.** The Stanza dependency parser component is based on a bidirectional LSTM architecture. It uses pretrained word embeddings, frequent word and lemma embeddings, character-level word embeddings, summed XPOS and UPOS embeddings, and summed FEATS embeddings as inputs. The average UAS across treebanks is reported to be at 77%. Considering only high-resource languages, this average rises to about 87%, while for low-resource languages it can be estimated to be no better than 70% (LAS of 63% plus a generous epsilon) (Qi et al., 2018). Nguyen (2020)

trained a dependency parser for Vietnamese following a very similar architecture and measured a UAS of 77%, which is in good alignment with the above figures.

**NLTK.** The NLTK has different dependency parsers available, but it is not easy to find out a whole lot of details about them. Judging from the NLTK-wiki it seems that—compared to SpaCy and Stanza—their approach is somewhat outdated (Bird, 2015).

None of the above tools use generative LLMs directly, but a lot of current research is aimed at figuring out how to leverage the power of LLMs to push the limits of dependency parsing accuracy. Due to their cross-lingual transfer capabilities, these new approaches promise to be especially valuable in the low-resource domain as demonstrated by Liu et al. (2026) using a self-optimization algorithm. Their study finds that depending on the low-resource language, models in the 7B to 8B range achieve average unlabeled attachment scores of 65% to 83%. Most interestingly, using Qwen2.5-7B-instruct they report a UAS of around 75% for Vietnamese, which is on-par (but notably not better than) the more traditional BiLSTM-based tools.

Further benchmarking results of different models and systems can be found at [nlpprogress.com](https://nlpprogress.com).

### 3 Our Approach

In this study, an agent is defined as a system that combines a language model with callable external tools and can select relevant tools during dependency parsing. The underlying principle is that low-resource languages are not necessarily resource-free. Dictionaries, grammars, partial annotations, and small treebanks may still provide useful linguistic evidence. Access to these resources can support more informed predictions, resembling how a human analyst consults reference materials when examining an unfamiliar language.

In this study we use treebanks as a source of supplementary information and although the project addresses low-resource settings, the use of treebanks as reference sources does not assume that complete depen-

dependency annotations are generally available. Dependency parsing is commonly performed after tokenization, lemmatization, part-of-speech tagging, and morphological analysis. UPOS and FEATS may therefore be available before dependency structures are assigned. These intermediate annotations can support the morphology and retrieval tools. Sentence retrieval can also benefit from any amount of verified parsing data. A small seed set of completed analyses may already provide relevant examples for sentences with similar lexical or syntactic patterns. The training treebanks are therefore used as bounded sources of linguistic evidence and as controlled reference data for evaluation, not as an assumption that all target sentences are already annotated.

#### 3.1 Agentic Tools

The project uses a shared dependency parsing pipeline to compare direct parsing with tool-supported parsing across multiple languages. Sentences are represented as JSON-style lists of token dictionaries at the pipeline boundary and as Stanza Document and Sentence objects internally. This keeps CoNLL-U input, tool arguments, and returned annotations consistent across languages.

Four reusable tools provide linguistic evidence and structural checks. The tools are designed in a way so that they can be applied to Vietnamese, German, Taiwanese, Marathi, and Low Saxon, or really all other UD treebanks. This design avoids separate implementations for individual languages.

**Statistical Morphology Lookup Tool.** The tool creates a case insensitive index from each training treebank. For every input token, it returns observed LEMMA, UPOS, and FEATS candidates with frequency counts. The result provides statistical, language specific morphological evidence through one common interface.

**Surface Form and Part of Speech Retrieval Tool.** This method compares a query sentence with training

sentences through surface form and UPOS sequence overlap. It returns the highest scoring examples with FORM, LEMMA, UPOS, FEATS, HEAD, and DEPREL annotations.

**Bag of Words Retrieval Tool.** This method combines multiset word overlap with sentence length similarity. It provides a complementary baseline for example selection and returns complete annotations, including dependency heads and relations.

**Dependency Tree Validation Tool.** The tool checks whether head indices are valid, whether each tree contains exactly one root, and whether the dependency structure is acyclic. Invalid structures can therefore be identified before evaluation.

## 4 Experiment

The Universal Dependencies (UD) parsing pipeline was evaluated in both a direct setting (without tools) and an agentic setting, in which the model could invoke external tools when required. This section details the data collection and methodology of the NLP pipeline.

### 4.1 Data collection and processing

Five Universal Dependencies (UD) treebanks were chosen to span a range of resource levels: English, Marathi, Taiwanese, Low\_Saxon, and Vietnamese. Although the project initially had only 4 lower-resource languages, English as a benchmark for the evaluation was also added at a later stage. Taiwanese, being a lower-resource language, did not have the official UD site, so the converted version of Sinica Treebank, one of the major Mandarin Chinese treebanks, to Universal Dependencies was implemented (Hsieh et al., 2022). The columns were rearranged so that the data matched the official UD format and could be called by the stanza NLP toolkit (Qi et al. 2020).

For each language, ten sentences were drawn from the official test split: sentences were sorted by character length, the list was divided into twenty equal-sized bins, and the sentence at the middle ten cut points were selected. This picks sentences at approximately the

30th, 35th, 40th, ..., 75th percentiles of the length distribution. This yields a length-stratified sample rather than a random one, so that both very short and comparatively long sentences are represented while avoiding the extremes.

### 4.2 Platform and dependencies

The following models were used in the experiment:

1. GGUF-quantized **Qwen3.5-9B** (Q4\_K\_M) (see HF model card: unsloth/Qwen3.5-9B-GGUF)
2. BitsAndBytes 4-bit NF4 quantization **Qwen3.5-9B** (see HF model card: Qwen/Qwen3.5-9B)

The reason for using two almost identical but ultimately different models was that, due to limited computational resources, the experiments had to be run in parallel on dissimilar hardware. To accommodate both inference modalities, the pipeline has two model wrappers:

- **LocalModel:** loads the model directly using the transformers library. This wrapper was used for running Qwen HF-compatible model.
- **LlamaCppModel:** This wrapper sends requests to a running llama.cpp server. The GGUF-quantized model was served locally using llama.cpp. The wrapper communicated with the llama.cpp server through its OpenAI-compatible interface.

### 4.3 Inference

In the experiment, the model is given a reduced parsing task: for every token in a sentence, predict HEAD (the id of its syntactic governor or 0 if the token is the root) following UD v2 conventions. The model receives the sentence text and a JSON array of {id, form} token objects, and is required to return a JSON array of {id, head} objects with no additional text. There are two inference modalities that are compared to one another:

**Direct prompting** (baseline): The model receives the DIRECT\_PARSING\_SYSTEM\_PROMPT and no tools. It must produce the head assignment for the whole sentence in a single pass of generation, relying only on its parametric knowledge.

**Agentic parsing:** The model receives the AGENTIC\_PARSING\_SYSTEM\_PROMPT, which is identical to the direct-prompting prompt except that it instructs the model to prefer tool calls over extended chain-of-thought reasoning - and is given access to the four tools described in the previous section.

In both conditions, both system prompts explicitly discourage the model from restating or re-deriving its analysis and instruct it not to hedge (“wait”, “actually”, “let me reconsider”), to keep generations concise; the agentic prompt additionally frames tool use as a substitute for, rather than a supplement to, long chains of thought.

The agent is allowed up to 10 iterations of generation/tool-execution per sentence and up to 10,000 new tokens per generation call (`max_iters_per_call`, `max_new_tokens`); sampling is enabled with temperature of 1, `top_p` of 0.95, `top_k` of 20 and repetition penalty of 1.1.

Tool calls are parsed from the model output in either JSON or a pseudo-XML (`<tool_call>/<function=...>`) format, executed against the corresponding Python implementation, and the result is appended to the conversation as a tool message before the next generation step. The full message trace for every sentence—system prompt, user turn, all assistant/tool turns, and the final parse—is logged to a per-sentence JSON file for later analysis.

The entire agentic NLP pipeline is run on 50 sentences, 10 from each language. For every language and inference modality it took approximately 1 hour to run the HF Qwen model using a A100 GPU. Using consumer hardware (Intel Core Ultra 7 H155) and the unsloth version of the that model, the same process took approximately 4 hours.

#### 4.4 Evaluation Method

Predicted and gold trees are compared for each sentence across all languages using unlabeled attachment score (UAS): the proportion of tokens whose predicted

head matches the gold head, computed per sentence and aggregated per language. Four additional diagnostics are recorded alongside UAS:

- whether the sentence was parsed successfully at all (i.e. the model returned a well-formed head assignment rather than leaving heads unset),
- whether the gold tree is projective (to allow later analysis of whether projectivity predicts parsing difficulty),
- the number of tool calls made during parsing (always 0 for the direct-prompting condition) and the number of calls to each of the four tools,
- how many tokens the model generated to produce the final output.

## 5 Results

## 6 Conclusion

## 7 Author Contribution Statement

Everyone explains the extend of their contribution in their own words.

Dang Nhat Quang. Expand...

Han Dai

Nikhila Gadge

Jan Raring

## Bibliography

- Bird, S. (2015, ). *Dependency Parsing* · *nlk/nltk Wiki*. <https://github.com/nltk/nltk/wiki/Dependency-Parsing>
- Explosion AI. (2026a, ). *DependencyParser* · *spaCy API Documentation*. <https://spacy.io/api/dependencyparser>
- Explosion AI. (2026b, ). *Facts & Figures* · *spaCy Usage Documentation*. <https://spacy.io/usage/facts-figures>
- Honnibal, M., & Johnson, M. (2014). Joint Incremental Disfluency Detection and Dependency Parsing. *Transactions of the Association for Computational Linguistics*, 2, 131–142. [https://doi.org/10.1162/tac1\\_a\\_00171](https://doi.org/10.1162/tac1_a_00171)

- Liu, J., Li, Y., Yu, Z., Huang, Y., & Gao, S. (2026). Improving cross-lingual dependency parsing via LLM-based transferring and self-optimizing synthetic data augmentation. *Expert Systems with Applications*, 315, 131600. <https://doi.org/https://doi.org/10.1016/j.eswa.2026.131600>
- Nguyen, L. (2020). Implementing Bi-LSTM-based deep biaffine neural dependency parser for Vietnamese Universal Dependency Parsing. In H. T. M. Nguyen, X.-S. Vu, & C. M. Luong (Eds.), *Proceedings of the 7th International Workshop on Vietnamese Language and Speech Processing: Proceedings of the 7th International Workshop on Vietnamese Language and Speech Processing*. <https://aclanthology.org/2020.vlsp-1.12/>
- Qi, P., Dozat, T., Zhang, Y., & Manning, C. D. (2018). Universal Dependency Parsing from Scratch. In D. Zeman & J. Hajič (Eds.), *Proceedings of the CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies: Proceedings of the CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies*. <https://doi.org/10.18653/v1/K18-2016>